

การวิเคราะห์อัลกอริทึม (ส่วนที่ 2)

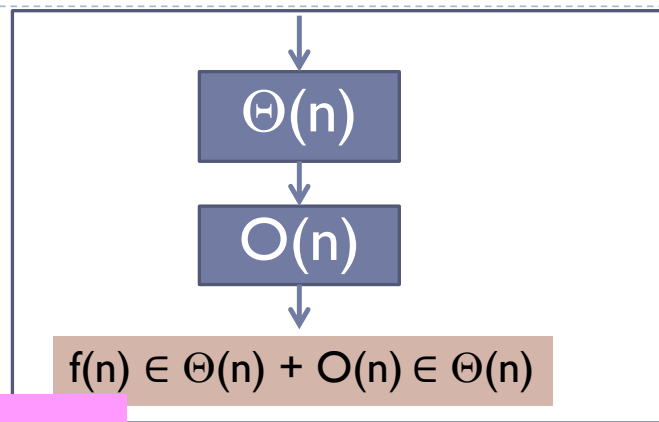
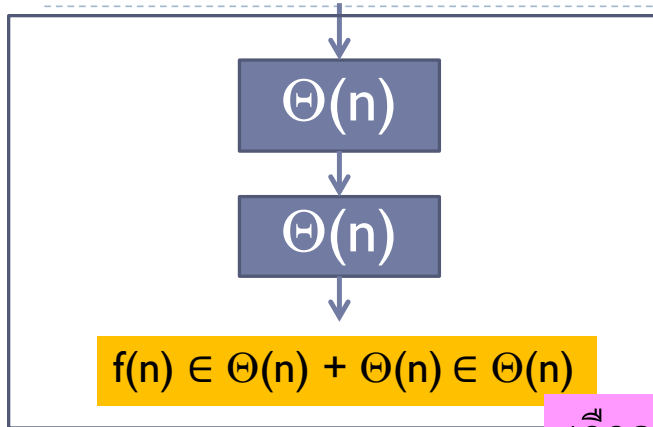
ผศ.จรรยา สายนุ้ย

344-111 ชุติวิชาการโปรแกรมและขั้นตอนวิธี

Time complexity : เลือก function ที่ biggest

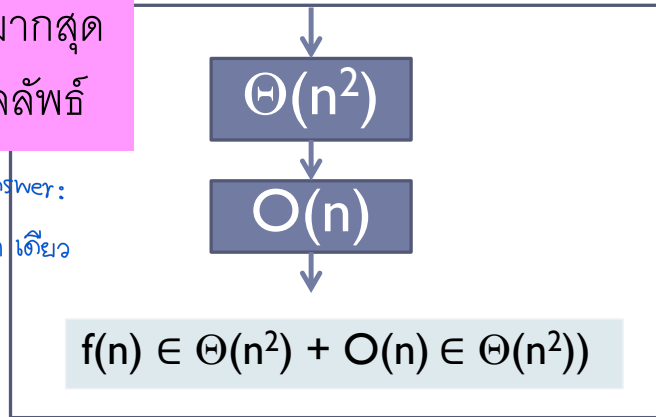
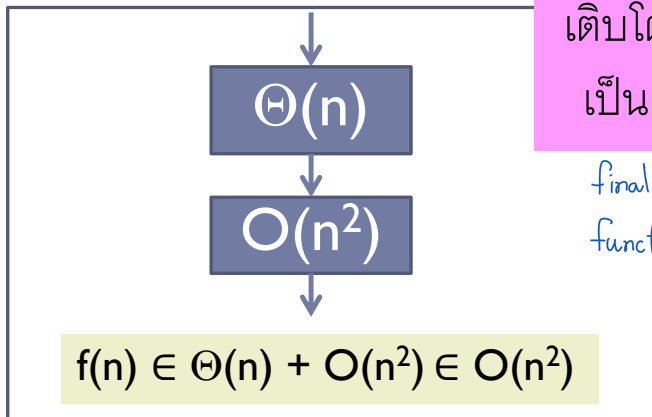
- ▶ การวิเคราะห์รูปแบบการทำงาน
 - 1. การทำงานแบบลำดับ (Sequence)
 - 2. คำสั่ง if...then...else
 - 3. For Loop
 - 4. While Loop
 - 5. Recursive Call

การทำงานแบบลำดับ

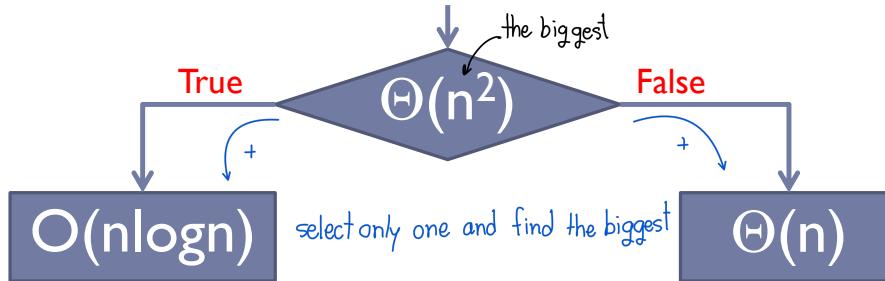


เลือกอัตราการเติบโตมากที่สุด
เป็นผลลัพธ์

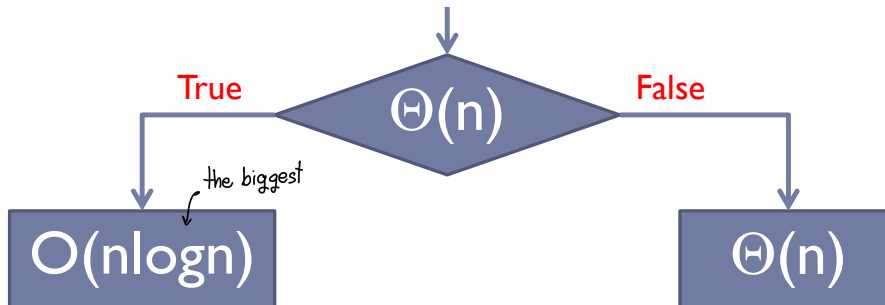
final answer:
function เดียว



if...then...else



$$f(n) \in \Theta(n^2) + O(n \log n) \in \Theta(n^2)$$



$$f(n) \in \Theta(n) + O(n \log n) \in O(n \log n)$$

For Loop

- ▶ ประสิทธิภาพตามจำนวนรอบ
- ▶ ใช้เวลาเท่ากับจำนวนรอบคูณเวลาที่ทำงานในลูป

```
for(i=1; i<=n; i++){  
    process(k);  
}
```

} n round

Process(k) ใช้เวลา $f(i)$

} $f(i) \times n = \sum_{i=1}^n f(i)$

$$\sum_{i=1}^n f(i)$$

For Loop

► Example I

```
for(i=1 to n)
```

```
  for(j=1 to n)
```

```
    s = s + a[i][j]
```

$\Theta(1)$

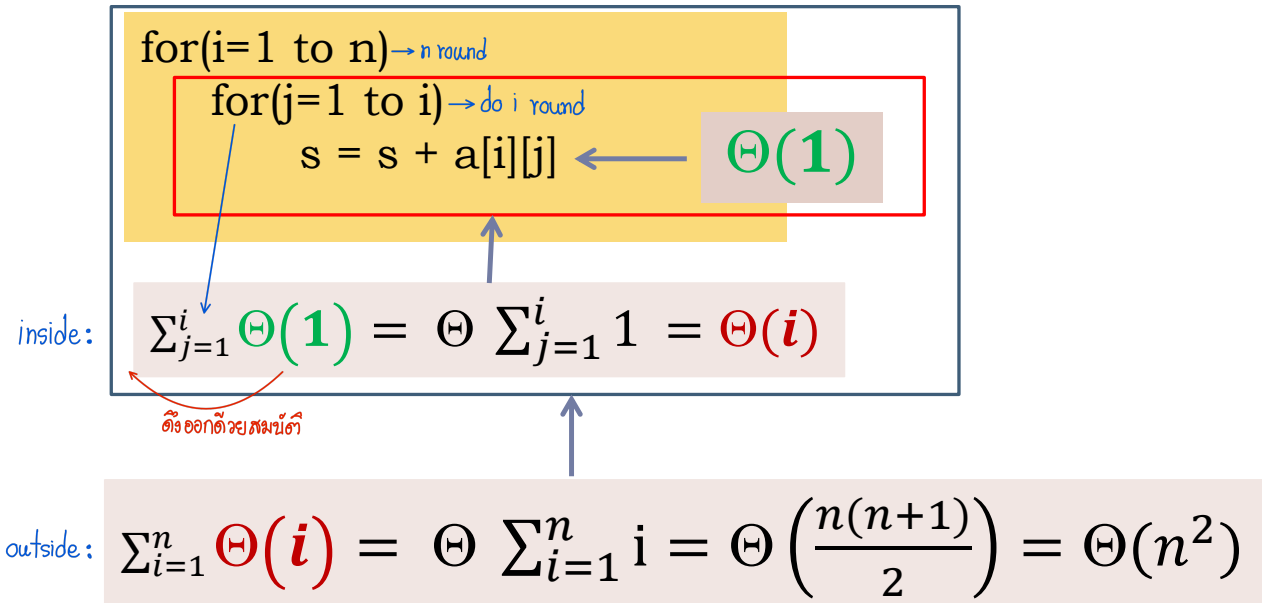
$$\sum_{j=1}^n \Theta(1) = \Theta \sum_{j=1}^n 1 = \Theta(n)$$

$$\sum_{i=1}^n \Theta(n) = \Theta \sum_{i=1}^n n = \Theta(n^2)$$



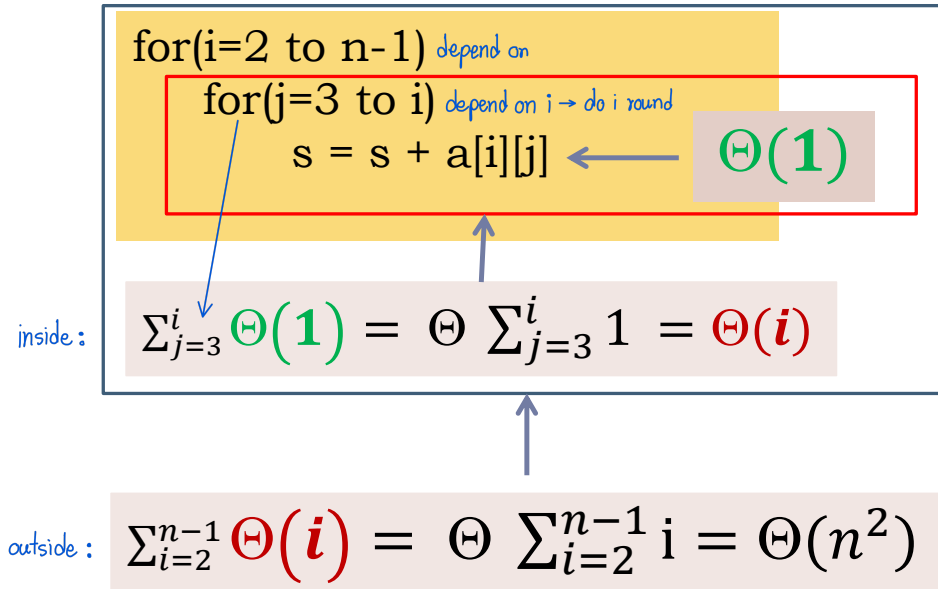
For Loop

► Example 2



For Loop

► Example 3



While Loop

▶ Example 1

```
while(n>0)
  n = ⌊n/2⌋ ←  $\Theta(1)$ 
```

$$\sum_{i=1}^{\log n} \Theta(1) = \Theta \sum_{i=1}^{\log n} 1 = \Theta(\log n)$$

- ▶ พิจารณาว่า คำสั่งในลูป **while** ทำงานกี่รอบ
- ▶ การที่ **n** ลดลงทีละครึ่ง รอบที่ **k** จะเหลือข้อมูล $n/2^k$
- ▶ ในกรณีนี้ ทำงานในลูปจนกระทั่ง **n = 1**
- ▶ ดังนั้น รอบที่ทำให้ **n = 1** ก็คือ $k = \log_2 n$
- ▶ ดังนั้นเวลาการทำงานจึงเป็น $\Theta(\log_2 n)$

While Loop

▶ Example 2

```
i=1
j=n
while( i < j )
    i = i + 3
    j = j - 5
```

$$\sum_{i=1}^{\frac{n}{8}} \Theta(1) = \Theta \sum_{i=1}^{\frac{n}{8}} 1 = \Theta(n)$$

6 ครั้ง

$$\Theta(1)$$

3 → 1 8 ← n = เข้าหากันทีละ 8

- ▶ i และ j จะเขยิบเข้าหากันทีละ 8
- ▶ แสดงว่า จำนวนรอบในลูป มีค่าอย่างมาก $n/8$ รอบ
- ▶ ดังนั้น จะใช้เวลาการทำงานเป็น $\frac{n}{8} \in \Theta(n)$

While Loop

► Example 3

```
for(j=2 to n)  
  key = a[j]  
  i = j-1
```

```
  while (i>0 and a[i] > key)
```

```
    a[i+1] = a[i]  
    i = i - 1
```

```
  a[i+1] = key
```

$\Theta(1)$

ความน่าจะเป็นที่จะพบมากที่สุด

$O(i) = O(j)$

$$\sum_{j=2}^n O(j) = O \sum_{j=2}^n j = O(n^2)$$

Exercise

```
Algorithm1(data[1..n], n){  
  for(i=1; i<n; i++){  
    tmp = data[i];  
    for(j=i+1; j<n; j++){  
      if(data[j]<tmp){  
        tmp = data[j];  
        k=j;  
      }  
    }  
    tmp = data[i];  
    data[i]=data[k];  
    data[k]=tmp;  
  }  
}
```

n round

$i=1$, $n-2$ round
 $i=2$, $n-3$ round
 $i=n-1$, 0 round
 $\downarrow$
 $(n-1) + (n-2) + (n-3) + \dots + 1 = \frac{n(n-1)}{2}$
 $= \frac{n^2}{2} - \frac{n}{2}$

Ans $O(n^2)$

การเขียนฟังก์ชันในรูปของสัญกรณ์เชิงเส้นกำกับ

- ▶ ผลบวกของพจน์หลายพจน์ เลือกพจน์ที่มีอัตราการเติบโตมากที่สุด
- ▶ ไม่สนใจค่าคงที่ เช่น $2n^2 \in \Theta(n^2)$



Recursive Call

- ▶ **Recursive call** นั้นเป็นลักษณะการทำงานแบบที่มีที่เรียกใช้ฟังก์ชันแบบซ้ำๆ และคำตอบของฟังก์ชันตัวหลังจะเป็นคำตอบของตัวก่อนหน้า

Factorial (n)

```
{  
    if (n = 0) or (n = 1)  
        return 1 ;  
    else  
        return n* Factorial(n-1)  
}
```

กรณี ที่ $n = 0$ หรือ 1

เวลาในการทำงาน

$$T(n) = C_1 \text{ (เป็นค่าคงที่ที่กำหนดขึ้นมา)}$$

กรณี ที่ $n \neq 0$ หรือ 1

เวลาในการทำงาน

$$T(n) = T(n-1) + C_2$$

(C_2 เป็นเวลาในการคูณของคำสั่ง else)

Recursive Call

► Binary search

```
bsearch(a[1..n], key, left, right){
```

```
    if (left > right) return -1
```

```
    mid =  $\lfloor (\text{left} + \text{right})/2 \rfloor$ 
```

```
    if (key = a[mid]) return mid
```

```
    if (key < a[mid])
```

```
        return bsearch(a, key, left, mid-1)
```

```
    else
```

```
        return bsearch(a, key, mid+1, right)
```

```
}
```

Base case:

$T(0) = ?$

Base case:

$T(n) = ?$

Analysis of binary search

$$t(n) = t\left(\frac{n}{2}\right) + \Theta(1)$$

$$t(n) = \left(t\left(\frac{n}{2^2}\right) + \Theta(1)\right) + \Theta(1) = t\left(\frac{n}{2^2}\right) + \Theta(1) + \Theta(1)$$

$$t(n) = \left(t\left(\frac{n}{2^3}\right) + \Theta(1)\right) + \Theta(1) + \Theta(1)$$

$$= t\left(\frac{n}{2^3}\right) + \Theta(1) + \Theta(1) + \Theta(1)$$

...

$$t(n) = t\left(\frac{n}{2^k}\right) + \Theta(1) + \cdots + \Theta(1) = \sum_{i=1}^{k+1} \Theta(1) \in \Theta(k) =$$

$$\Theta(\log n) \text{ กรณีหาไม่เจอ (กรณีทั่วไป = } O(\log n))$$

$$\frac{n}{2^{k-1}} = 1$$

$$n = 2^{k-1}$$

$$k - 1 = \log n$$

$$k = \log n + 1$$

Other examples: analysis of recursive algorithm

$$t(n) = 2 \left(t\left(\frac{n}{2}\right) \right) + n,$$

$$t(n) = 2 \left(2 \left(t\left(\frac{n}{2^2}\right) \right) + \frac{n}{2} \right) + n = 2^2 t\left(\frac{n}{2^2}\right) + n + n$$

$$t(n) = 2^2 \left(2 t\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \right) + n + n = 2^3 t\left(\frac{n}{2^3}\right) + n + n + n$$

...

$$t(n) = 2^k t\left(\frac{n}{2^k}\right) + n + \cdots + n$$

$$t(n) = \sum_{k=0}^{\log_2 n} n \in \Theta(n \log n)$$

Exercise:

```
hanoi (n, a, b, c){  
    if(n==0) return
```

```
    ① hanoi(n-1, a, b, c)
```

```
    1. print("move", n, a, "->" c)
```

```
    ② hanoi(n-1, b, a, c)
```

```
}
```

T(n) = ?

Recursive: $T(n) = 2T(n-1) + 1 = ?$

$$= 2(2T(n-2) + 1) + 1$$

$$= 2^2 T(n-2) + 2 + 1$$

$$= 2^2 (2T(n-3) + 1) + 2 + 1$$

$$= 2^3 T(n-3) + 2^2 + 2 + 1$$

$\vdots$

$$= 2^n T(n-n) +$$

Exercise: หาผลเฉลย (รูปแบบปิด) ของฟังก์ชัน $t(n)$

$$t(n) = 2t(n-1) + 1, t(0) = 1$$

Master method

▶ กรณี $t(n) = at(n/b) + \Theta(n^k)$ และให้ $c = \log_b a$

▶ จะได้ว่า

▶ $t(n) = \Theta(n^k)$ ถ้า $n^k > n^c$

▶ $t(n) = \Theta(n^k \log n)$ ถ้า $n^k = n^c$

▶ $t(n) = \Theta(n^c)$ ถ้า $n^k < n^c$

Exercise: ใช้ Master method หาผลเฉลยต่อไปนี้

1. $t(n) = t(n/2) + 1 = \Theta(n^{\log_2 1}) = \Theta(\log n)$
 $a=1, b=2, c=\log_2 1 = 1 = 1$
2. $t(n) = 2t(n/2) + 1 = \Theta(n)$
 $a=2, b=2, c=\log_2 2 = 1$
3. $t(n) = 2t(n/2) + n^2 = \Theta(n^2)$
 $a=2, b=2, c=\log_2 2 = 1$
4. $t(n) = 2t(n/2) + n = \Theta(n \log n)$
 $a=2, b=2, c=\log_2 2 = 1$
5. $t(n) = 2t(n/3) + n = \Theta(n)$
 $a=2, b=3, c=\log_3 2 \approx 0.63 < 1$

กรณี $t(n) = at(n/b) + \Theta(n^k)$ และ ให้ $c = \log_b a$

จะได้ว่า

- ▶ $t(n) = \Theta(n^k)$ ถ้า $n^k > n^c$
- ▶ $t(n) = \Theta(n^k \log n)$ ถ้า $n^k = n^c$
- ▶ $t(n) = \Theta(n^c)$ ถ้า $n^k < n^c$

Worst-Case, Best-Case, Average-Case Efficiencies

- ▶ Algorithm: *SequentialSearch*(A[0..n-1], key)

```
i = 0
While (i < n and A[i] ≠ key)
    i = i + 1
if (i < n) return i
else return -1
```

- ▶ **Worst-Case:** กรณีที่ $A[n-1] = \text{key}$

- ▶ วนลูป n รอบ

- ▶ **Best-Case:** กรณีที่ $A[0] = \text{key}$

- ▶ วนลูป 1 รอบ

- ▶ **Average-Case:** $\frac{1}{n} (1 + 2 + 3 + \dots + n) = \frac{n(n+1)}{2n} = \Theta(n)$

Binary search

```
bsearch(a[1..n], key, left, right){  
    if (left > right) return -1  
    mid =  $\lfloor (left + right)/2 \rfloor$   
    if (key = a[mid]) return mid  
    if( key < a[mid])  
        return bsearch(a, key, left, mid-1)  
    else  
        return bsearch(a, key, mid+1, right)  
}
```

- ▶ **Worst-Case:** $\Theta(\log(n))$
- ▶ **Best-Case:** $\Theta(1)$
- ▶ **Average-Case:** $\frac{1}{n} (1 + 2 \cdot 2^1 + 3 \cdot 2^2 + \dots + k \cdot 2^{k-1}) = \Theta(\log(n))$

Insertion sort

```
insertionSort(data[1..n], n){  
    for(i=2; i<=n; i++){  
        tmp = data[i];  
        j=i;  
        while(j > 1 && data[j-1]>tmp){  
            data[j] = data[j-1];  
            j=j-1;  
        }  
        data[j]=tmp;  
    }  
}
```

- ▶ **Worst-Case:** $\Theta(n^2)$
- ▶ **Best-Case:** $\Theta(n)$
- ▶ **Average-Case:** $\sum_{i=2}^n \left(\sum_{j=2}^i \frac{j}{i} \right) = \Theta(n^2)$

อ้างอิง

- ▶ สมชาย ประสิทธิ์จุตระกูล, การออกแบบและวิเคราะห์อัลกอริทึม
- ▶ Anany Levitin, Introduction to the Design and Analysis of Algorithm, second edition.

